

Official Report: PDF Static Analysis Script

Author: Solomon Mwaura

Date: April 27, 2026

Subject: Development and Implementation of a Python Script for PDF Static Analysis

1. Introduction

This report documents the development of `pdf_analysis.py`, a Python script designed to perform static analysis on suspicious PDF files. As a junior security analyst, the objective is to build a foundational tool that can automatically extract critical information, compute cryptographic hashes, and identify suspicious indicators within a PDF, thereby aiding in the initial triage of potentially malicious documents. This script serves as a practical exercise in forensic analysis, simulating real-world cybersecurity challenges.

2. Script Overview:

pdf_analysis.py

The `pdf_analysis.py` script is structured into several modular functions, each responsible for a specific aspect of the static analysis process. This modular design enhances readability, maintainability, and reusability.

2.1. Core Functionalities

The script performs the following key operations:

1. **File Reading:** Safely reads the target PDF file in binary mode.
2. **String Extraction:** Identifies and extracts readable ASCII strings from the binary content.
3. **Hash Computation:** Generates MD5 and SHA-256 cryptographic hashes for integrity verification and identification.
4. **Suspicious Indicator Detection:** Scans extracted strings against a predefined list of keywords commonly associated with PDF exploits.
5. **Report Generation:** Compiles all findings into a concise, formatted report.

3. Detailed Script Components

3.1. `read_file(file_path)`

This function is responsible for opening the specified PDF file in binary read mode (`'rb'`). It includes robust error handling to gracefully manage scenarios where the file might not exist (`FileNotFoundError`) or other I/O issues occur (`IOError`),

preventing script crashes and providing informative feedback to the user.

```
def read_file(file_path):
    try:
        with open(file_path, 'rb') as file:
            return file.read()
    except FileNotFoundError:
        print(f"Error: The file '{file_path}' was not found.")
        return None
    except IOError as e:
        print(f"Error reading file '{file_path}': {e}")
        return None
```

3.2. extract_strings (data)

This function employs regular expressions to search for sequences of printable ASCII characters within the binary data of the PDF. A minimum length of 4 characters is enforced to filter out noise. The function attempts to decode strings using UTF-8 first, falling back to ASCII with error ignoring to preserve as much information as possible while handling various encodings.

```
def extract_strings(data):
    pattern = rb"[\x20-\x7E]{4,}"
    raw_strings = re.findall(pattern, data)
    decoded_strings = []
    for s in raw_strings:
        try:
            decoded_strings.append(s.decode('utf-8'))
        except UnicodeDecodeError:
            decoded_strings.append(s.decode('ascii', errors='ignore'))
    return decoded_strings
```

3.3. compute_hashes (data)

Cryptographic hashing is a fundamental step in digital forensics for file identification and integrity verification. This function calculates both the MD5 and SHA-256 hashes of the entire PDF file's binary content, providing unique digital fingerprints for the document.

```
def compute_hashes(data):  
    md5_hash = hashlib.md5(data).hexdigest()  
    sha256_hash = hashlib.sha256(data).hexdigest()  
    return md5_hash, sha256_hash
```

3.4. detect_suspicious (strings)

This function scans the list of extracted strings for a comprehensive set of keywords and patterns commonly associated with malicious PDF activities. The expanded keyword list aims to detect various forms of embedded scripts, actions, and obfuscation techniques often found in exploit documents.

```
def detect_suspicious(strings):  
    keywords = [  
        "javascript", "action", "submit", "malicious", "alert",  
        "/js", "/openaction", "/launch", "/uri", "/acroform",  
        "eval(", "unescape(", "goto", "objstm", "xrefstm",  
        "encrypt", "flatedecode", "/annots", "/richmedia",  
        # ... (extensive list of PDF-specific keywords)  
    ]  
    found_indicators = []  
    for s in strings:  
        if any(key in s.lower() for key in keywords):  
            found_indicators.append(s)  
    return found_indicators
```

3.5. generate_report(file_path, md5, sha256, all_strings, suspicious, output_format)

The reporting function consolidates all analysis results into a clear and structured format. It supports both a human-readable text output (printed to console) and a machine-readable JSON output, making the script versatile for both manual analysis and integration into automated security workflows.

```
def generate_report(file_path, md5, sha256, all_strings, suspicious, output_format):
    report_data = {
        "file_path": file_path,
        "md5_hash": md5,
        "sha256_hash": sha256,
        "total_strings_extracted": len(all_strings),
        "suspicious_indicators_found": len(suspicious),
        "suspicious_indicators": suspicious,
        "strings_preview": all_strings[:5]
    }
    if output_format == 'json':
        return json.dumps(report_data, indent=4)
    else:
        # ... (text report generation logic)
        return report_text
```

3.6. Main Execution Block

The main block orchestrates the execution flow, utilizing `argparse` for command-line argument parsing to specify the target PDF file and desired output format. It ensures that functions are called in the correct sequence and that the final report is displayed or returned appropriately.

```
def main():
    parser = argparse.ArgumentParser(description="Perform static analysis")
    parser.add_argument("file_path", help="Path to the PDF file to analyze")
    parser.add_argument("-o", "--output", choices=['text', 'json'], default='text', help="Output format")
    args = parser.parse_args()
    data = read_file(args.file_path)
    if data:
        md5, sha256 = compute_hashes(data)
        all_strings = extract_strings(data)
        suspicious = detect_suspicious(all_strings)
        report = generate_report(args.file_path, md5, sha256, all_strings)
        print(report)
    else:
        print("Analysis could not be completed.")
```

4. Conclusion

The `pdf_analysis.py` script provides a robust and efficient solution for the static analysis of suspicious PDF files. By integrating memory-safe file handling, comprehensive string extraction, cryptographic hashing, and an extensive list of suspicious indicators, this tool empowers junior security analysts to quickly triage and gain initial insights into potentially malicious documents. Its modular design and flexible reporting options make it a valuable asset in any cybersecurity toolkit for proactive threat detection and incident response.

5. Attachments

- `pdf_analysis.py`: The Python script for PDF static analysis.